

Feature Matching

Aadith Warriar

What is Feature Matching?

Feature matching proposes image measurements that may correspond to the same physical scene point.

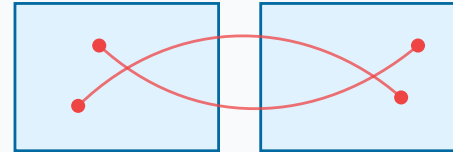
$$\mathbf{x}_i \leftrightarrow \mathbf{x}'_j$$



Why Feature Matching Matters

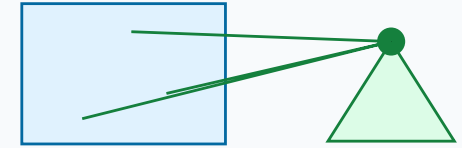
- Panorama stitching estimates image warps
- Visual localization estimates camera pose
- SfM and SLAM recover cameras and 3D points
- Calibration and stereo vision rely on verified correspondences

Panorama stitching



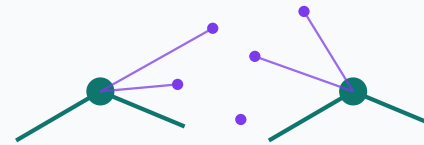
matches estimate an image warp

Visual localization



2D image points + 3D map points recover pose

Structure from Motion



tracks across views become cameras and points

SLAM / tracking



persistent correspondences stabilize motion estimates

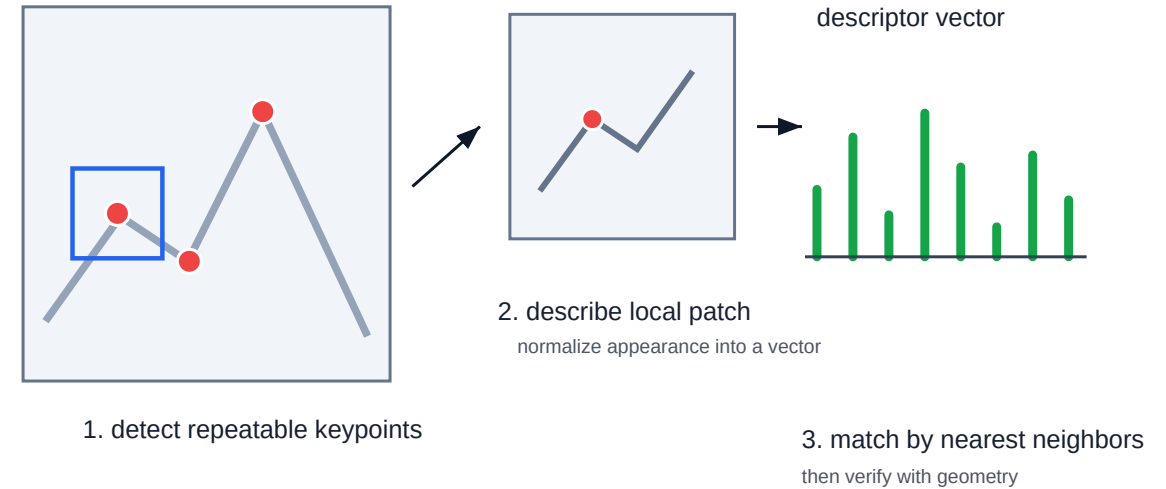
Feature Matching Pipeline

Feature matching usually means:

1. detect repeatable keypoints
2. compute local descriptors
3. match descriptors
4. reject outliers

The detector answers “where”; the descriptor answers “what does the neighborhood look like?”

Feature matching decomposes correspondence



Types: Classical and Learned

Classical

- Hand-crafted keypoints and descriptors
- Detects corners/blobs using fixed image operators
- Matching via nearest-neighbor descriptor search
- Fast, interpretable, and data-independent
- Struggles with repetitive patterns, lighting, and low texture
- e.g SIFT, ORB, SURF

Learned

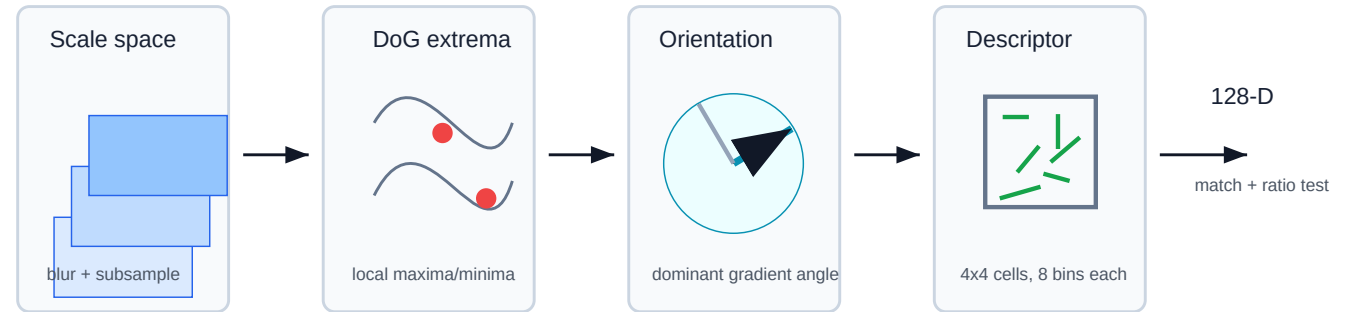
- Features learned directly from data using Neural Networks
- Match refinement, Dense correspondence
- More robust to viewpoint, illumination, and weak texture
- Higher accuracy, but requires training data and more compute
- e.g SuperPoint, SuperGlue, ROMA, MicKey, MAST3R

Classical Methods: SIFT Intuition

SIFT is a hand-designed answer to invariance.

- scale: search extrema in scale space
- rotation: align to dominant local gradient
- illumination: use normalized gradient histograms
- ambiguity: ratio test before geometric verification

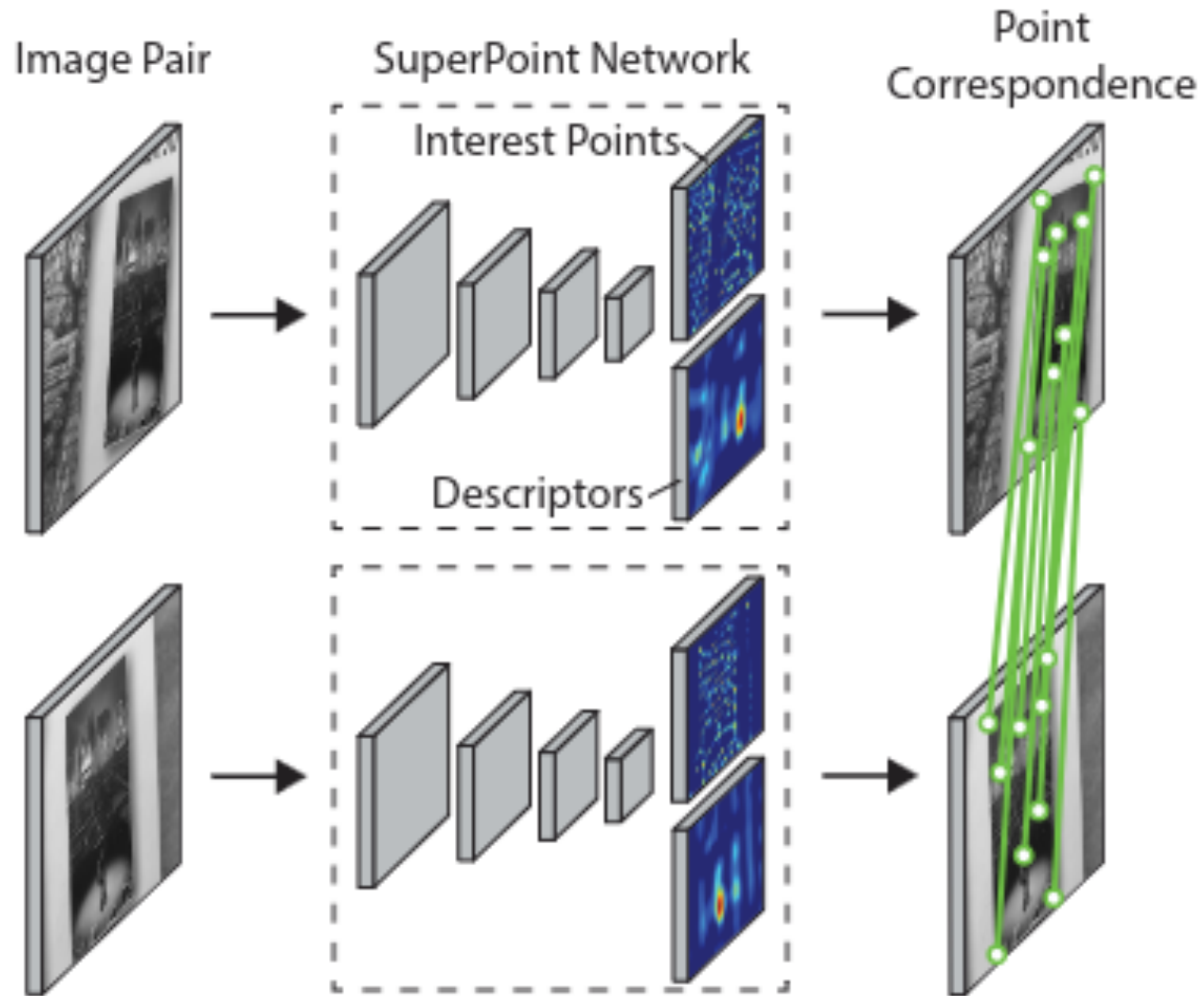
SIFT: engineered invariance pipeline



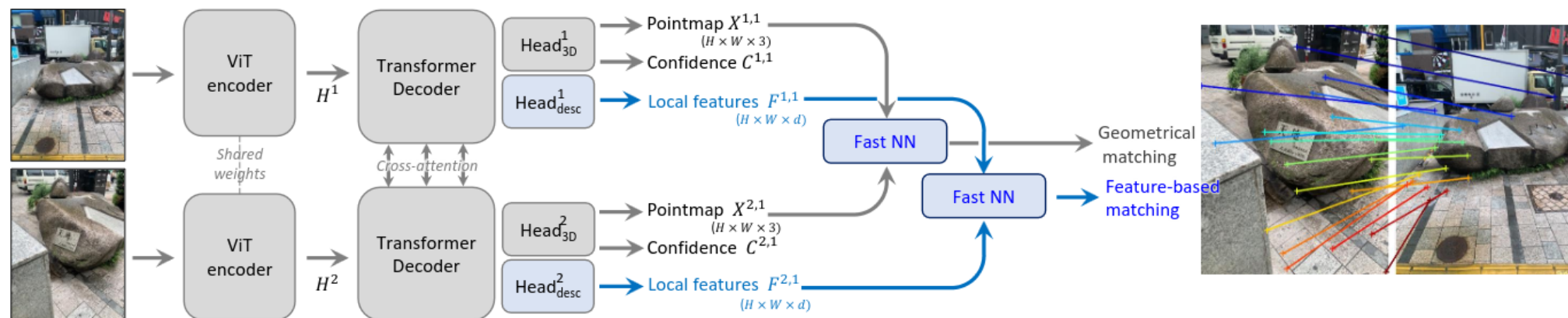
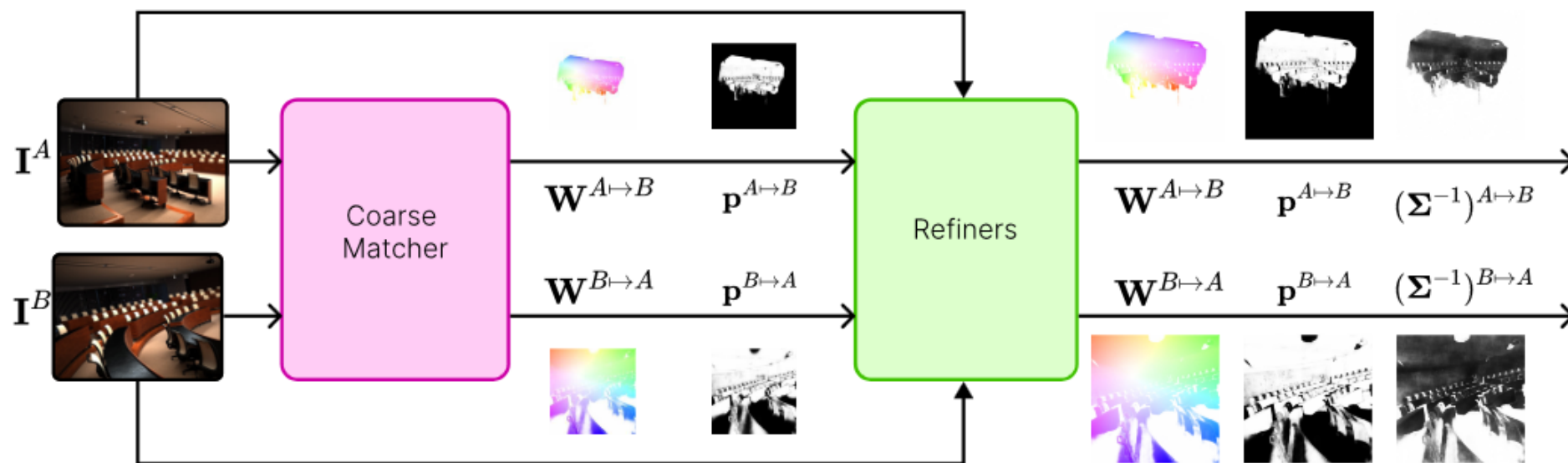
Limitations

- low texture: few stable gradients
- repetitive structure: many descriptors look alike
- large viewpoint change: local affine deformation breaks the descriptor
- non-Lambertian effects: specularities and shadows violate appearance constancy

Learnt Feature Matching



Learnt Feature Matching



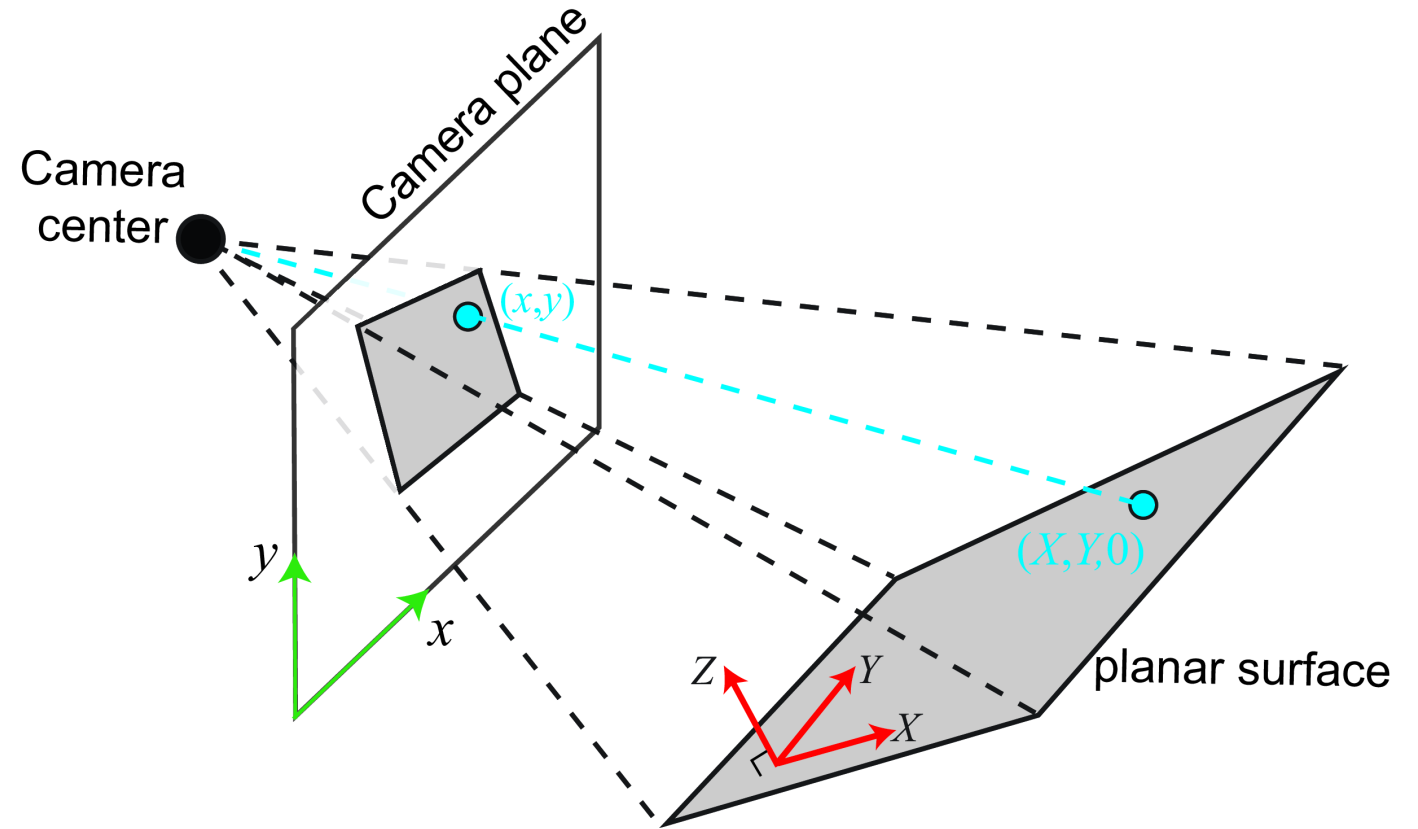
Homography: Image-to-Image Geometry

A homography maps points between two images:

$$\mathbf{x}' \sim H\mathbf{x}, \quad H \in \mathbb{R}^{3 \times 3}$$

It is valid for:

- a planar scene,
- pure camera rotation,
- or image rectification / warping.



Homography from a Plane

To derive the homography relation for planar scenes, we begin with the standard camera projection equation:

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

where:

- $\mathbf{K}$ is the camera intrinsic matrix,
- $\mathbf{R}$ and $\mathbf{t}$ are the camera rotation and translation,
- (X, Y, Z) is a 3D world point,
- (x, y) is the corresponding image point.

For a planar scene, we choose the world coordinate system such that all points on the plane satisfy:

$$Z = 0$$

Substituting this into the projection equation gives:

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix}$$

Let the columns of the rotation matrix $\mathbf{R}$ be:

$$\mathbf{R} = [\mathbf{c}_1 \quad \mathbf{c}_2 \quad \mathbf{c}_3]$$

Then the projection equation can be rewritten as:

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{c}_1 \quad \mathbf{c}_2 \quad \mathbf{t}] \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Since the product of the two 3×3 matrices is itself a 3×3 matrix, we define:

$$\mathbf{H} = \mathbf{K} \begin{bmatrix} \mathbf{c}_1 & \mathbf{c}_2 & \mathbf{t} \end{bmatrix}$$

Therefore, the projection equation reduces to:

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

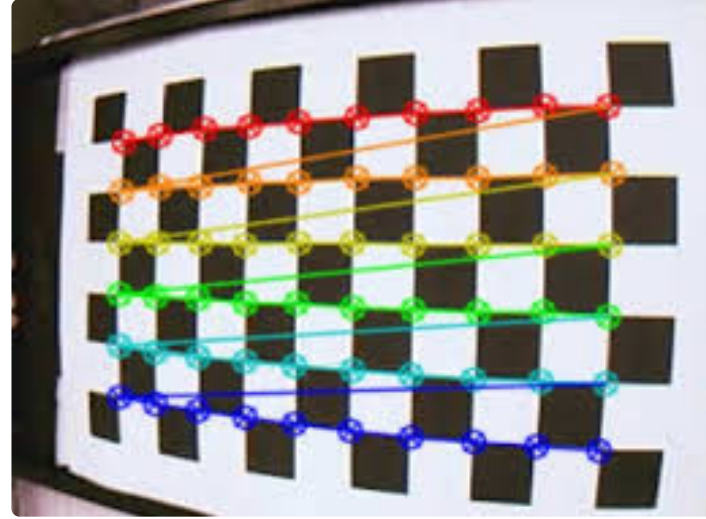
This shows that points lying on a plane are related to their image coordinates through a homography transformation $\mathbf{H}$.

Zhang's Calibration Method

For a planar checkerboard with $Z = 0$:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Each view provides constraints on K .



Pipeline

1. detect corners
2. estimate one H per view
3. solve for $B = K^{-\top} K^{-1}$
4. refine all parameters by reprojection error

Where do $\mathbf{r}_1$ and $\mathbf{r}_2$ come from?

For each checkerboard image, we estimate a homography:

$$\mathbf{H} = \mathbf{K} \begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t} \end{bmatrix}$$

Let:

$$\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 & \mathbf{h}_2 & \mathbf{h}_3 \end{bmatrix}$$

Then:

$$\mathbf{h}_1 = \lambda \mathbf{K} \mathbf{r}_1$$

$$\mathbf{h}_2 = \lambda \mathbf{K} \mathbf{r}_2$$

$$\mathbf{h}_3 = \lambda \mathbf{K} \mathbf{t}$$

where λ is a scale factor.

Since $\mathbf{H}$ is estimated directly from point correspondences on the checkerboard, the vectors $\mathbf{h}_1$ and $\mathbf{h}_2$ are known.

The unknowns are:

- the intrinsic matrix $\mathbf{K}$,
- the rotation vectors $\mathbf{r}_1, \mathbf{r}_2$,
- the translation vector $\mathbf{t}$.

The key insight is that $\mathbf{r}_1$ and $\mathbf{r}_2$ come from a rotation matrix, so they satisfy:

$$\mathbf{r}_1^\top \mathbf{r}_2 = 0$$

and

$$\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$$

Substituting:

$$\mathbf{r}_1 = \lambda \mathbf{K}^{-1} \mathbf{h}_1$$

$$\mathbf{r}_2 = \lambda \mathbf{K}^{-1} \mathbf{h}_2$$

gives constraints involving only $\mathbf{K}$ and the known homography columns.

$$\mathbf{r}_1^T \mathbf{r}_2 = 0$$

This allows Zhang's method to solve for the intrinsic parameters using multiple checkerboard views.

Fundamental Matrix: Uncalibrated Geometry

For uncalibrated cameras, image points are represented directly in pixel coordinates:

$$\mathbf{x}, \quad \mathbf{x}'$$

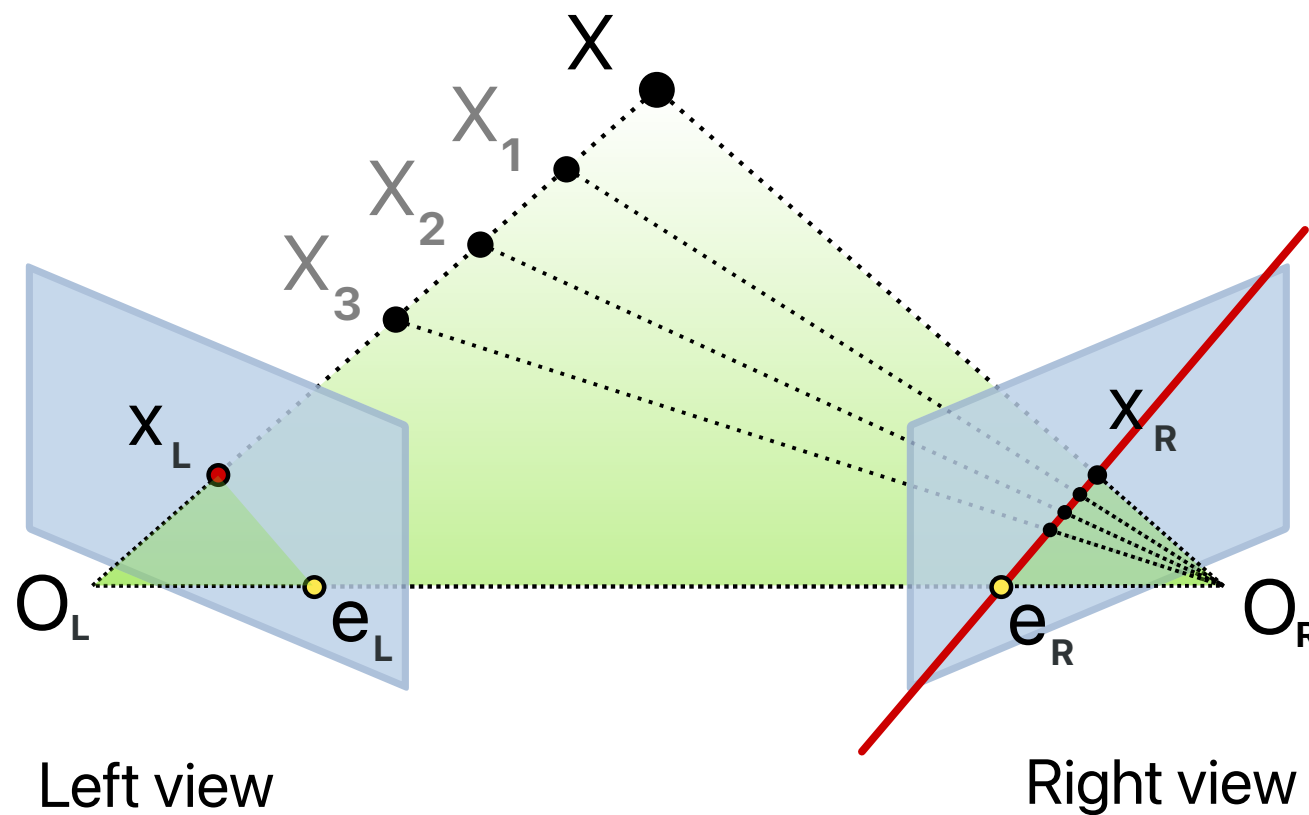
A point in one image maps to an epipolar line in the other:

$$\ell' = \mathbf{F}\mathbf{x}$$

The corresponding point must lie on that line:

$$\mathbf{x}'^\top \mathbf{F}\mathbf{x} = 0$$

This is the **epipolar constraint**.



Epipolar Geometry

A 3D point together with the two camera centers defines an **epipolar plane**.

This plane intersects each image plane along an epipolar line.

Therefore:

- a point in one image constrains the correspondence in the other image,
- correspondence search reduces from 2D to a 1D line search.

All epipolar lines intersect at the **epipole**.

Fundamental Matrix Properties

The fundamental matrix satisfies:

$$\mathbf{F} \in \mathbb{R}^{3 \times 3}, \quad \text{rank}(\mathbf{F}) = 2, \quad \mathbf{F} \sim \alpha \mathbf{F}$$

where α is an arbitrary nonzero scale factor.

The epipoles are the null vectors of $\mathbf{F}$:

$$\mathbf{F} \mathbf{e} = \mathbf{0}, \quad \mathbf{F}^\top \mathbf{e}' = \mathbf{0}$$

$\mathbf{F}$ depends on both camera intrinsics and relative camera motion, and operates directly on homogeneous pixel coordinates.

8-Point Algorithm

Given a correspondence:

$$\mathbf{x} = (u, v, 1)^\top, \quad \mathbf{x}' = (u', v', 1)^\top$$

the epipolar constraint expands to:

$$u'uf_{11} + u'vf_{12} + u'f_{13} + v'uf_{21} + v'vf_{22} + v'f_{23} + uf_{31} + vf_{32} + f_{33} = 0$$

Each correspondence contributes one linear equation in the unknown entries of $\mathbf{F}$.

Stacking all correspondences gives:

$$A\mathbf{f} = \mathbf{0}$$

where:

[Math Processing Error]

and each row of A is:

[Math Processing Error]

Solve for $\mathbf{f}$ using SVD by taking the right singular vector corresponding to the smallest singular value.

Rank-2 Constraint

The least-squares solution generally produces a full-rank matrix. However, a valid fundamental matrix must satisfy:

$$\text{rank}(\mathbf{F}) = 2$$

Compute the SVD:

$$\hat{\mathbf{F}} = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^\top$$

Then enforce the rank constraint by setting the smallest singular value to zero:

$$\mathbf{F} = U \text{diag}(\sigma_1, \sigma_2, 0) V^\top$$

Normalized 8-Point Algorithm

Before solving, apply similarity transforms:

$$\tilde{\mathbf{x}} = T\mathbf{x}, \quad \tilde{\mathbf{x}}' = T'\mathbf{x}'$$

Choose T, T' so each point set has centroid at the origin and average distance $\sqrt{2}$.

After estimating $\tilde{\mathbf{F}}$:

$$\mathbf{F} = T'^{\top} \tilde{\mathbf{F}} T$$

Normalization significantly improves numerical conditioning and is essential in practice.

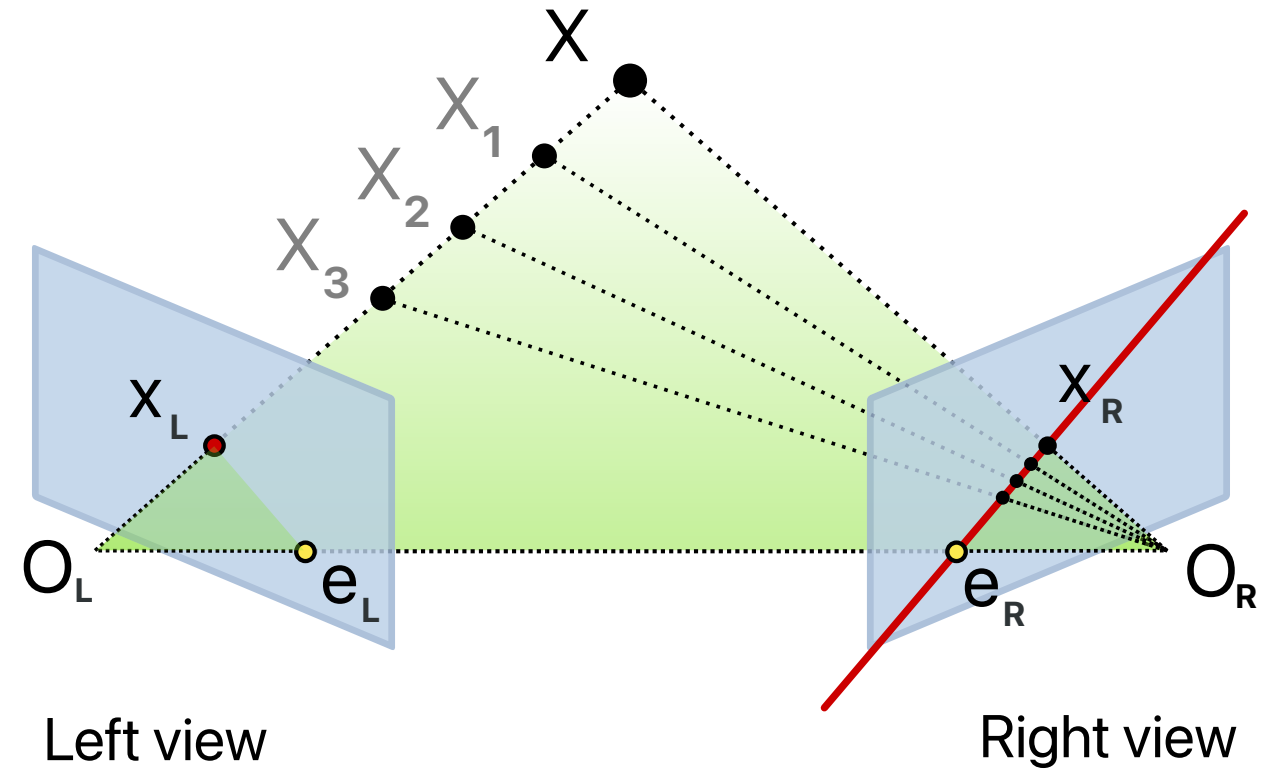
Essential Matrix: Calibrated Geometry

For calibrated cameras, image points are represented in normalized camera coordinates:

$$\hat{\mathbf{x}} = K^{-1}\mathbf{x}, \quad \hat{\mathbf{x}}' = K'^{-1}\mathbf{x}'$$

The two camera frames are related by:

$$\mathbf{X}' = \mathbf{R}\mathbf{X} + \mathbf{t}$$



Coplanarity Constraint

The vectors:

$$\hat{\mathbf{x}}', \quad \mathbf{t}, \quad \mathbf{R}\hat{\mathbf{x}}$$

all lie in the same epipolar plane.

Therefore their scalar triple product must vanish:

[Math Processing Error]

Using the skew-symmetric cross-product matrix:

[Math Processing Error]

we write:

$$\mathbf{t} \times \mathbf{x} = [\mathbf{t}]_{\times} \mathbf{x}$$

Essential Matrix Definition

Substituting the cross-product matrix gives:

$$\hat{\mathbf{x}}'^{\top} [\mathbf{t}]_{\times} \mathbf{R} \hat{\mathbf{x}} = 0$$

Define the essential matrix:

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

Then the epipolar constraint becomes:

$$\hat{\mathbf{x}}'^{\top} \mathbf{E} \hat{\mathbf{x}} = 0$$

The essential matrix therefore encodes the relative pose between two calibrated cameras.

Essential Matrix Properties

The essential matrix satisfies:

$$\text{rank}(\mathbf{E}) = 2$$

because:

$$\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$$

where:

- $[\mathbf{t}]_{\times}$ has rank 2,
- $\mathbf{R}$ has rank 3.

Its singular values satisfy:

$$\sigma_1 = \sigma_2, \quad \sigma_3 = 0$$

Thus:

$$\mathbf{E} = U \text{diag}(1, 1, 0) V^{\top}$$

up to scale.

Relationship to the Essential Matrix

For calibrated cameras:

$$\hat{\mathbf{x}} = K^{-1}\mathbf{x}, \quad \hat{\mathbf{x}}' = K'^{-1}\mathbf{x}'$$

The normalized-coordinate epipolar constraint is:

$$\hat{\mathbf{x}}'^{\top} \mathbf{E} \hat{\mathbf{x}} = 0$$

Substituting normalized coordinates:

$$(K'^{-1}\mathbf{x}')^{\top} \mathbf{E} (K^{-1}\mathbf{x}) = 0$$

Rearranging:

$$\mathbf{x}'^{\top} K'^{-\top} \mathbf{E} K^{-1} \mathbf{x} = 0$$

Define the fundamental matrix:

$$\boxed{\mathbf{F} = K'^{-\top} \mathbf{E} K^{-1}}$$

Thus:

- $\mathbf{E}$ operates in normalized camera coordinates,
- $\mathbf{F}$ operates directly in pixel coordinates.

Recovering Camera Motion

Given $\mathbf{E}$, compute:

$$\mathbf{E} = U \operatorname{diag}(1, 1, 0) V^{\top}$$

The decomposition yields:

$$\mathbf{R}, \mathbf{t}$$

up to a global scale ambiguity.

This enables:

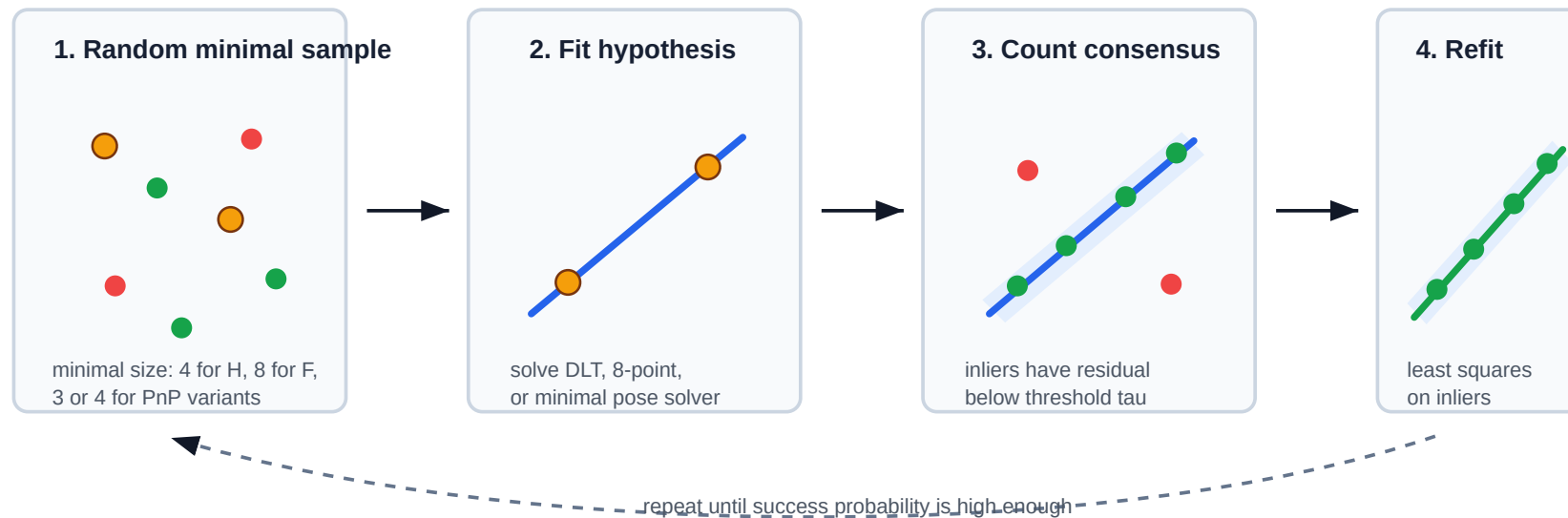
- stereo reconstruction,
- visual odometry,
- SLAM,
- structure-from-motion.

RANSAC

RANSAC repeatedly:

1. samples a minimal subset,
2. fits a candidate model,
3. counts inliers under a residual threshold,
4. refits using the best consensus set.

RANSAC: estimate from minimal samples, trust consensus



Use the residual for the model: transfer error for H, epipolar distance for F/E, reprojection error for pose.

RANSAC Iteration Count

If w is the inlier probability, s is the minimal sample size, and p is the desired success probability:

$$N \geq \frac{\log(1 - p)}{\log(1 - w^s)}$$

Interpretation:

- w^s is the probability one random sample is all inliers,
- $1 - w^s$ is the probability one sample fails,
- $(1 - w^s)^N$ is the probability all N samples fail.

Perspective-n-Point (PnP)

Given:

- camera intrinsics K ,
- 3D points $\mathbf{X}_i$,
- image observations $\mathbf{x}_i$,

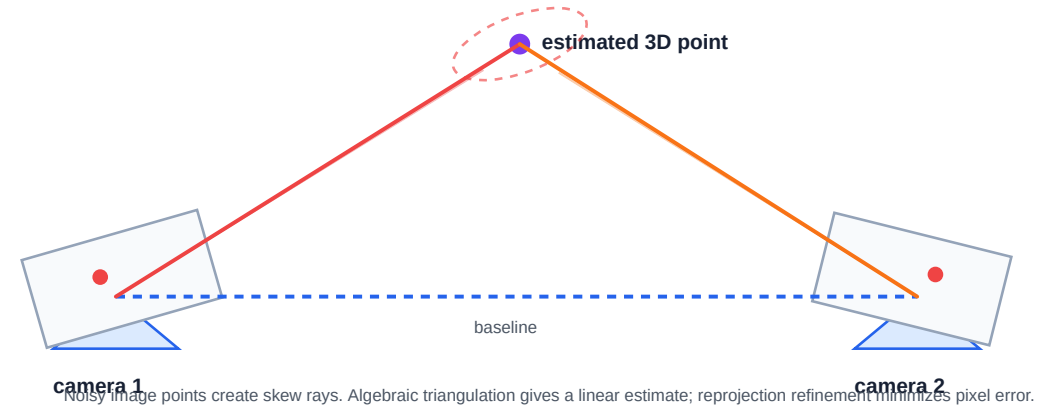
PnP estimates the camera pose:

$$R, \mathbf{t}$$

such that:

[Math Processing Error]

Triangulation: recover depth from intersecting viewing rays



PnP Intuition

PnP finds the camera position and orientation that best aligns projected 3D points with observed image points.

The objective is reprojection error:

$$\min_{R, \mathbf{t}} \sum_i \|\pi(K(R\mathbf{X}_i + \mathbf{t})) - \mathbf{x}_i\|_2^2$$

Common PnP Solvers

- **P3P / AP3P**: minimal solvers used inside RANSAC
- **EPnP**: efficient for many correspondences
- iterative refinement further reduces reprojection error

PnP vs DLT

PnP assumes a calibrated camera and estimates:

$$R, \mathbf{t}$$

DLT instead estimates a general projection matrix:

$$P \in \mathbb{R}^{3 \times 4}$$

without enforcing rotation constraints.

PnP estimates calibrated camera pose.

DLT estimates a general projective camera matrix.

Triangulation

Given two camera matrices:

$$P_1, P_2$$

and matched image points:

$$\mathbf{x}_1, \mathbf{x}_2$$

triangulation estimates the 3D point:

$$\mathbf{X}$$

that produced the observations.

Geometrically, triangulation finds the intersection of viewing rays from multiple cameras.

With noisy observations, the rays rarely intersect exactly.

Linear Triangulation

Use the projective constraint:

$$\mathbf{x}_i \times P_i \mathbf{X} = 0$$

Each view contributes linear equations of the form:

$$u_i P_{i,3}^\top - P_{i,1}^\top = 0$$

$$v_i P_{i,3}^\top - P_{i,2}^\top = 0$$

Stack all equations into:

$$A\mathbf{X} = 0$$

and solve using SVD.

Triangulation Objective

In practice, triangulation minimizes reprojection error:

$$\min_{\mathbf{X}} \sum_i \|\pi(P_i \mathbf{X}) - \mathbf{x}_i\|_2^2$$

Good triangulation requires:

- low reprojection error,
- positive depth,
- sufficient camera baseline.

The math works when the coordinate convention is explicit and the residual matches the model being estimated.

References

- Hartley, R. and Zisserman, A. *Multiple View Geometry in Computer Vision*, 2nd edition.
- Lowe, D. “Distinctive Image Features from Scale-Invariant Keypoints.” IJCV, 2004.
- DeTone, D., Malisiewicz, T., Rabinovich, A. “SuperPoint: Self-Supervised Interest Point Detection and Description.” CVPR Workshops, 2018.
- Edstedt, J. et al. “RoMa: Robust Dense Feature Matching.” CVPR, 2024.
- OpenCV documentation: `findHomography`, `findFundamentalMat`, `findEssentialMat`, `recoverPose`, and `solvePnP`.
- Zhang, Z. “A Flexible New Technique for Camera Calibration.” PAMI, 2000.
- Epipolar geometry SVG: Arne Nordmann, Wikimedia Commons, CC BY-SA / GFDL.
- Feature matching example image: Georgia Tech CS4476 project result, Fall 2016.
- Zhang checkerboard image: downloaded from the user-provided image URL.
- Foundations of Computer Vision - <https://visionbook.mit.edu/>